Hard - #4 - Median of Two Sorted Arrays

⚠ Problem Statement - Given two sorted arrays "nums 1" & "nums 2", find the median of the two arrays combined

The Solution must run in O(log (m+n)) time.

⚠ Main Problem Solution - Instead of merging the arrays, use "binary search" to find ~~that~~ a partition that divides both arrays into :-

"LEFT | RIGHT"

the partition is correct when :- "left 1 <= right 2"
"left 2 <= right 1"

Once Found :- • If total elements are odd, median = largest element on the left.

• If total elements are even, median = average of largest left & smallest right.

⚠ Brute Force Method - • Merge both sorted arrays & find the middle element.

• This takes O(m+n) time & O(m+n) space, which is inefficient because the problem requires O(log (m+n)) time.

M T W T F S S

Page No.: YOUVA

Date:

Algorithm:

1. Make "nums1" the smaller array.

2. Calculate the number of elements that should be on the left :-

"leftSize = (m + n + 1) / 2"

3. Binary search for the partition in "nums1".

4. Find the partition in "nums2" :-

"j = leftSize - i"

5. Find the four boundary values :-

"left1" ; "right1" ; "left2" ; "right2"

6. Check if the partition is valid :-

"left1 <= right2 && left2 <= right1"

7. If "left1 > right2", move the partition first.

8. Otherwise, move the partition right.

9. If the partition is correct :- • Odd → "max(left1, left2)"

• Even → "(max(left1, left2) + min(right1, right2)) / 2.0"

10. Use "INT_MIN" & "INT_MAX" when a partition reaches the beginning or end of an array.

M T W T F S S

Page No.: YOUVA

Date:

▲ Important Points :- • Always binary search on the smaller array.

• "`j = leftSize - i`" keeps the left side balanced.

• Correct Partition :- "`left1 <= right2 && left2 <= right1`"

• "`INT_MIN`" & "`INT_MAX`" handle empty left/right parts.

• Use "`/2.0`" to get a decimal result for even-sized arrays.

▲ Complexity :- Time :- O(log(min(m, n))

Space :- O(1)